

03_ingest-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ ingest.py
```

El archivo `ingest.py` se encuentra dentro de la carpeta de rutas de la API:

```
backend/app/api/routes/
```

Este archivo define el endpoint principal de entrada de eventos del laboratorio SIEM MVP:

```
POST /ingest
```

Su función no se limita a guardar eventos. También ejecuta el flujo principal del laboratorio:

1. Recibir un evento.
2. Validarlo mediante `IngestPayload`.
3. Guardarlo como `Event`.
4. Calcular una clave de agrupación.
5. Consultar reglas activas.
6. Evaluar criterios de coincidencia.
7. Aplicar `throttle`.
8. Aplicar anti-duplicado.
9. Aplicar `threshold`.
10. Crear alertas si corresponde.
11. Confirmar la transacción.

Por tanto, este archivo es uno de los más importantes del backend, porque concentra el flujo funcional principal del SIEM.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' backend/app/api/routes/ingest.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

Ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from __future__ import annotations

from datetime import datetime, timezone, timedelta

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
from app.schemas.event import EventOut
from app.schemas.ingest import IngestPayload

router = APIRouter(prefix="/ingest", tags=["ingest"])

def _compute_group_key(ev: Event) -> str | None:
    if not ev.meta:
        return None
    return ev.meta.get("host")

@router.post("", response_model=EventOut)
def ingest(payload: IngestPayload, db: Session = Depends(get_db)):
    now = datetime.now(timezone.utc)

    try:
        # 1) Guardar evento
        ev = Event(
            ts=now,
            source=payload.source,
            severity=payload.severity,
            message=payload.message,
            meta=payload.meta,
        )
        db.add(ev)
        db.flush() # ev.id disponible

        group_key = _compute_group_key(ev)

        # 2) Evaluar reglas habilitadas
        rules = db.execute(
            select(Rule).where(Rule.enabled.is_(True)).order_by(Rule.id.asc())
        ).scalars().all()

        for rule in rules:
            # 2.1 Source
            if rule.source and ev.source != rule.source:
                continue

            # 2.2 Severity
            if rule.severity_min is not None and ev.severity < rule.severity_min:
                continue

            # 2.3 Contains
```

```

if rule.contains and rule.contains.lower() not in (ev.message or "").lower():
    continue

# 2.4 Meta match exacto
if rule.meta_match:
    if not ev.meta:
        continue
    if any(ev.meta.get(k) != v for k, v in rule.meta_match.items()):
        continue

# 3) Throttle (ignorando closed)
# Decisión: si group_key es None, NO aplicamos throttle (no hay agrupación fiable)
if (
    group_key is not None
    and rule.throttle_seconds is not None
    and rule.throttle_seconds > 0
):
    last_alert_ts = (
        db.execute(
            select(Alert.created_at)
            .where(
                Alert.rule_id == rule.id,
                Alert.group_key == group_key,
                Alert.status.in_("open", "ack"),
            )
            .order_by(Alert.created_at.desc())
            .limit(1)
        )
        .scalar_one_or_none()
    )
    if last_alert_ts:
        delta = (now - last_alert_ts).total_seconds()
        if delta < rule.throttle_seconds:
            continue

# 3.5) Anti-duplicado (si hay open/ack, no crear otra)
# Decisión: si group_key es None, NO aplicamos anti-duplicado (no hay agrupación fiable)
if group_key is not None:
    existing_active_alert_id = (
        db.execute(
            select(Alert.id)
            .where(
                Alert.rule_id == rule.id,
                Alert.group_key == group_key,
                Alert.status.in_("open", "ack"),
            )
            .order_by(Alert.created_at.desc())
            .limit(1)
        )
        .scalar_one_or_none()
    )
    if existing_active_alert_id is not None:
        continue

# 4) Threshold
if rule.threshold_count is not None and rule.threshold_seconds is not None:
    if group_key is None:
        continue

    window_start = now - timedelta(seconds=rule.threshold_seconds)
    stmt = select(func.count(Event.id)).where(Event.ts >= window_start)

    if rule.source:
        stmt = stmt.where(Event.source == rule.source)

```

```

        if rule.severity_min is not None:
            stmt = stmt.where(Event.severity >= rule.severity_min)

        if rule.contains:
            stmt = stmt.where(Event.message.ilike(f"%{rule.contains}%"))

        if rule.meta_match:
            stmt = stmt.where(Event.meta.contains(rule.meta_match))

        stmt = stmt.where(Event.meta.contains({"host": group_key}))

        matched_count = db.execute(stmt).scalar_one()
        if matched_count < rule.threshold_count:
            continue

    # 5) Crear alerta
    alert = Alert(
        rule_id=rule.id,
        event_id=ev.id,
        title=f"Rule matched: {rule.name}",
        group_key=group_key,
    )
    db.add(alert)

    db.commit()
    return ev

except Exception as e:
    db.rollback()
    raise HTTPException(status_code=500, detail="Ingest failed") from e

```

4 Función general del archivo

El archivo `ingest.py` define la ruta principal de ingesta del laboratorio.

El endpoint expuesto es:

```
POST /ingest
```

Este endpoint recibe un payload con esta estructura:

```

{
  "source": "auth",
  "severity": 4,
  "message": "Failed login attempt",
  "meta": {
    "host": "server-01",
    "user": "admin"
  }
}

```

Después realiza el siguiente proceso:

1. Crea un evento `Event`.
2. Lo añade a la sesión de base de datos.
3. Obtiene su id con `db.flush()`.
4. Calcula `group_key` a partir de `meta.host`.
5. Recupera las reglas activas.
6. Evalúa cada regla contra el evento.
7. Si una regla coincide, comprueba `throttle`, `anti-duplicado` y `threshold`.
8. Si todo se cumple, crea una alerta `Alert`.
9. Ejecuta `commit`.
10. Devuelve el evento creado mediante `EventOut`.

Este archivo conecta varios módulos del proyecto:

```
schemas/ingest.py → valida la entrada
schemas/event.py  → define la salida
models/event.py   → almacena el evento
models/rule.py    → consulta reglas
models/alert.py   → genera alertas
db/session.py     → proporciona la sesión de base de datos
```

5 Estructura general del archivo

El archivo puede dividirse en ocho bloques:

```
from __future__ import annotations
```

Importación futura de anotaciones.

```
from datetime import datetime, timezone, timedelta
```

Importaciones para trabajar con fechas, UTC y ventanas temporales.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import func, select
from sqlalchemy.orm import Session
```

Importaciones de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
from app.schemas.event import EventOut
from app.schemas.ingest import IngestPayload
```

Importaciones internas del proyecto.

```
router = APIRouter(prefix="/ingest", tags=["ingest"])
```

Creación del router.

```
def _compute_group_key(ev: Event) -> str | None:
```

Función auxiliar para calcular la clave de agrupación.

```
@router.post("", response_model=EventOut)
def ingest(...):
```

Definición del endpoint principal.

```
try:
    ...
except Exception as e:
    ...
```

Control transaccional y manejo de errores.

Visualmente:

```
ingest.py
├─ Importaciones
```

```
├ Router /ingest
├ Función auxiliar _compute_group_key
└ Endpoint POST /ingest
  ├── Obtener fecha actual
  ├── Crear Event
  ├── Calcular group_key
  ├── Obtener reglas activas
  ├── Evaluar criterios
  ├── Aplicar throttle
  ├── Aplicar anti-duplicado
  ├── Aplicar threshold
  ├── Crear Alert
  ├── Commit
  └ Rollback si falla
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Activa el comportamiento moderno de Python para anotaciones de tipos.

En este archivo permite usar expresiones como:

```
str | None
```

en la función:

```
def _compute_group_key(ev: Event) -> str | None:
```

Esto indica que la función puede devolver una cadena o `None`.

Importación de fecha, zona horaria y ventanas temporales

```
from datetime import datetime, timezone, timedelta
```

Esta línea importa tres elementos del módulo estándar `datetime`.

datetime

Se usa para obtener el momento actual:

```
now = datetime.now(timezone.utc)
```

timezone

Se usa para indicar que la fecha debe generarse en UTC:

```
timezone.utc
```

Esto es importante en sistemas de eventos porque evita confusiones con zonas horarias locales.

timedelta

Se usa para calcular ventanas temporales:

```
window_start = now - timedelta(seconds=rule.threshold_seconds)
```

Esto permite evaluar reglas de threshold, por ejemplo:

```
5 eventos en los últimos 60 segundos
```

Importación de FastAPI

```
from fastapi import APIRouter, Depends, HTTPException
```

Esta línea importa tres elementos de FastAPI.

APIRouter

Permite crear un router independiente para las rutas de ingesta:

```
router = APIRouter(prefix="/ingest", tags=["ingest"])
```

Depends

Permite usar dependencias de FastAPI.

En este archivo se usa para obtener una sesión de base de datos:

```
db: Session = Depends(get_db)
```

HTTPException

Permite devolver errores HTTP controlados.

En este archivo se usa si falla la ingesta:

```
raise HTTPException(status_code=500, detail="Ingest failed") from e
```

Importación de SQLAlchemy

```
from sqlalchemy import func, select
```

Importa herramientas para construir consultas SQL.

func

Permite usar funciones SQL, como:

```
func.count(Event.id)
```

Esto se usa para contar eventos dentro de una ventana temporal.

select

Permite construir consultas SQLAlchemy.

Ejemplos del archivo:

```
select(Rule)
select(Alert.created_at)
```

```
select(Alert.id)
select(func.count(Event.id))
```

Importación de Session

```
from sqlalchemy.orm import Session
```

Importa el tipo `Session` de SQLAlchemy.

Se usa como anotación en:

```
db: Session = Depends(get_db)
```

Indica que `db` será una sesión de base de datos.

Importación de get_db

```
from app.db.session import get_db
```

Importa la función que proporciona sesiones de base de datos.

Esta función se analizó en:

```
02_Base-de-datos/03_Analisis-tecnico-de-base-de-datos/02_session-py
```

En este archivo permite que el endpoint use PostgreSQL sin abrir y cerrar conexiones manualmente.

Importación de modelos

```
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
```

Estas líneas importan los tres modelos principales del flujo:

```
Event → evento recibido
Rule → regla de detección
Alert → alerta generada
```

La relación funcional es:

```
Event + Rule → Alert
```

Importación de schemas

```
from app.schemas.event import EventOut
from app.schemas.ingest import IngestPayload
```

Importa los schemas de entrada y salida.

```
IngestPayload → valida el JSON recibido
EventOut → define la respuesta devuelta
```

Creación del router


```
router = APIRouter(prefix="/ingest", tags=["ingest"])
```

Crea un router con prefijo `/ingest` .

Desglose:

```
prefix="/ingest"
```

Todas las rutas de este archivo empezarán por `/ingest` .

```
tags=["ingest"]
```

Agrupar el endpoint en Swagger bajo la etiqueta `ingest` .

Como el decorador usa:

```
@router.post("")
```

la ruta final será:

```
POST /ingest
```

Función auxiliar `_compute_group_key`

```
def _compute_group_key(ev: Event) -> str | None:
```

Define una función auxiliar que calcula la clave de agrupación de un evento.

Desglose:

```
_compute_group_key
```

El guion bajo inicial indica que es una función interna del módulo.

```
ev: Event
```

Recibe un objeto `Event` .

```
-> str | None
```

Devuelve una cadena o `None` .

Comprobación de `meta`

```
if not ev.meta:  
    return None
```

Si el evento no tiene metadatos, no puede calcularse una clave de agrupación.

En ese caso devuelve:

```
None
```

Esto afectará a `throttle`, `anti-duplicado` y `threshold`, porque esas lógicas dependen de `group_key` .

Obtención de `host`

```
return ev.meta.get("host")
```

Si el evento tiene `meta` , la función intenta obtener la clave:

```
host
```

Ejemplo:

```
{
  "meta": {
    "host": "server-01"
  }
}
```

En ese caso:

```
group_key = "server-01"
```

Si `host` no existe, devuelve `None` .

Decorador del endpoint

```
@router.post("", response_model=EventOut)
```

Registra un endpoint HTTP de tipo `POST` .

Como el router tiene prefijo `/ingest` , la ruta final es:

```
POST /ingest
```

El parámetro:

```
response_model=EventOut
```

indica que la respuesta se serializará usando el schema `EventOut` .

Por tanto, aunque la función devuelva un objeto SQLAlchemy `Event` , FastAPI lo transformará en una respuesta JSON con los campos definidos en `EventOut` .

Definición de la función `ingest`

```
def ingest(payload: IngestPayload, db: Session = Depends(get_db)):
```

Define la función que se ejecuta al recibir una petición `POST /ingest` .

Parámetros:

```
payload → cuerpo JSON validado con IngestPayload
db       → sesión SQLAlchemy proporcionada por get_db
```

FastAPI hace automáticamente:

1. Lee el JSON recibido.
2. Lo valida con `IngestPayload`.
3. Ejecuta `get_db()`.
4. Pasa ambos valores a `ingest()`.

Fecha actual en UTC

```
now = datetime.now(timezone.utc)
```

Obtiene la fecha y hora actual en UTC.

Esta variable se usa para:

- Asignar `ts` al evento.
- Calcular ventanas temporales de `threshold`.
- Comparar con alertas anteriores para `throttle`.

Bloque `try`

```
try:
```

Inicia un bloque protegido.

Todo el flujo de ingesta se ejecuta dentro de este `try`.

Si ocurre cualquier error, se captura en el bloque `except`.

Esto permite ejecutar:

```
db.rollback()
```

para revertir cambios parciales.

Creación del evento

```
ev = Event(  
    ts=now,  
    source=payload.source,  
    severity=payload.severity,  
    message=payload.message,  
    meta=payload.meta,  
)
```

Crea un objeto `Event`.

Este objeto todavía no está confirmado en la base de datos.

Relación entre `payload` y modelo:

```
payload.source    → ev.source  
payload.severity  → ev.severity  
payload.message   → ev.message  
payload.meta      → ev.meta  
now               → ev.ts
```

Esta línea transforma el schema validado en un modelo ORM persistente.

Añadir evento a la sesión

```
db.add(ev)
```

Añade el objeto `Event` a la sesión `SQLAlchemy`.

Esto significa:

```
este objeto debe insertarse en la base de datos
```

Pero todavía no se ha ejecutado `commit`.

`db.flush()`

```
db.flush() # ev.id disponible
```

Envía las operaciones pendientes a la base de datos sin cerrar la transacción.

La finalidad principal es obtener el `id` del evento antes de hacer `commit`.

Esto es necesario porque más adelante se crea una alerta con:

```
event_id=ev.id
```

Sin `flush`, `ev.id` podría no estar disponible todavía.

Diferencia:

```
flush → sincroniza con la base de datos, pero no confirma definitivamente
commit → confirma la transacción
```

Cálculo de `group_key`

```
group_key = _compute_group_key(ev)
```

Calcula la clave de agrupación del evento.

La función busca:

```
ev.meta["host"]
```

Resultado posible:

```
"server-01"
None
```

El `group_key` se usa después para:

- throttle
- anti-duplicado
- threshold
- agrupación de alertas

Consulta de reglas habilitadas

```
rules = db.execute(
    select(Rule).where(Rule.enabled.is_(True)).order_by(Rule.id.asc())
).scalars().all()
```

Esta consulta obtiene todas las reglas activas.

Desglose:

```
select(Rule)
```

Selecciona objetos `Rule`.

```
.where(Rule.enabled.is_(True))
```

Filtra solo reglas activas.

```
.order_by(Rule.id.asc())
```

Ordena por `id` ascendente.

```
db.execute(...)
```

Ejecuta la consulta.

```
.scalars()
```

Extrae objetos `Rule` en lugar de filas SQL completas.

```
.all()
```

Devuelve todas las reglas en una lista.

Resultado conceptual:

```
rules = [rule1, rule2, rule3]
```

Bucle de evaluación de reglas

```
for rule in rules:
```

Recorre cada regla activa.

Por cada regla se comprueba si el evento cumple sus condiciones.

Si una condición no se cumple, se usa:

```
continue
```

para saltar a la siguiente regla.

Criterio `source`

```
if rule.source and ev.source != rule.source:
    continue
```

Si la regla tiene definido un origen (`rule.source`) y el evento no coincide con ese origen, la regla no aplica.

Ejemplo:

```
rule.source = "auth"
ev.source    = "firewall"
```

Resultado:

```
no coincide → continue
```

Si `rule.source` es `None` , no se filtra por origen.

Criterio `severity_min`

```
if rule.severity_min is not None and ev.severity < rule.severity_min:
    continue
```

Si la regla tiene una severidad mínima y el evento tiene menor severidad, la regla no aplica.

Ejemplo:

```
rule.severity_min = 5
ev.severity       = 3
```

Resultado:

```
3 < 5 → continue
```

Si `severity_min` es `None`, no se filtra por severidad.

Criterio `contains`

```
if rule.contains and rule.contains.lower() not in (ev.message or "").lower():
    continue
```

Comprueba si el mensaje del evento contiene un texto determinado.

Desglose:

```
rule.contains
```

Texto que debe aparecer.

```
rule.contains.lower()
```

Convierte el texto de la regla a minúsculas.

```
(ev.message or "").lower()
```

Convierte el mensaje del evento a minúsculas.

Esto hace que la búsqueda no distinga entre mayúsculas y minúsculas.

Ejemplo:

```
rule.contains = "failed login"
ev.message    = "Failed Login Attempt"
```

Resultado:

```
coincide
```

Criterio `meta_match`

```
if rule.meta_match:
    if not ev.meta:
        continue
    if any(ev.meta.get(k) != v for k, v in rule.meta_match.items()):
        continue
```

Evalúa coincidencias exactas sobre el campo `meta`.

Primero comprueba si la regla tiene `meta_match`.

Si la regla exige metadatos pero el evento no tiene `meta`, se descarta:

```
if not ev.meta:
```

```
continue
```

Después compara cada clave y valor:

```
ev.meta.get(k) != v
```

Ejemplo:

```
rule.meta_match = {  
    "user": "admin",  
    "action": "login_failed"  
}
```

El evento debe tener esos mismos valores en `meta`.

La función `any(...)` devuelve `True` si alguna clave no coincide.

Si alguna no coincide, se hace `continue`.

Bloque de throttle

```
if (  
    group_key is not None  
    and rule.throttle_seconds is not None  
    and rule.throttle_seconds > 0  
):
```

Este bloque aplica throttle si se cumplen tres condiciones:

1. Existe `group_key`.
2. La regla tiene `throttle_seconds`.
3. `throttle_seconds` es mayor que 0.

El throttle evita generar demasiadas alertas repetidas para la misma regla y grupo.

Consulta de última alerta activa

```
last_alert_ts = (  
    db.execute(  
        select(Alert.created_at)  
        .where(  
            Alert.rule_id == rule.id,  
            Alert.group_key == group_key,  
            Alert.status.in_(("open", "ack")),  
        )  
        .order_by(Alert.created_at.desc())  
        .limit(1)  
    )  
    .scalar_one_or_none()  
)
```

Esta consulta busca la fecha de la última alerta activa para la misma regla y grupo.

Filtros:

```
Alert.rule_id == rule.id  
Alert.group_key == group_key  
Alert.status in ("open", "ack")
```

Solo considera alertas:

```
open → abiertas
ack → reconocidas
```

Ignora alertas cerradas.

Ordena por fecha descendente:

```
.order_by(Alert.created_at.desc())
```

y toma solo una:

```
.limit(1)
```

Comparación con `throttle_seconds`

```
if last_alert_ts:
    delta = (now - last_alert_ts).total_seconds()
    if delta < rule.throttle_seconds:
        continue
```

Si existe una alerta anterior, calcula cuántos segundos han pasado.

```
delta = (now - last_alert_ts).total_seconds()
```

Si el tiempo transcurrido es menor que el throttle configurado, no crea otra alerta.

Ejemplo:

```
throttle_seconds = 300
última alerta hace 100 segundos
```

Resultado:

```
100 < 300 → continue
```

Bloque anti-duplicado

```
if group_key is not None:
```

Solo se aplica si existe `group_key`.

La decisión del código es clara:

```
si group_key es None, no se aplica anti-duplicado
```

Esto evita deduplicar alertas sin una agrupación fiable.

Consulta de alerta activa existente

```
existing_active_alert_id = (
    db.execute(
        select(Alert.id)
        .where(
            Alert.rule_id == rule.id,
            Alert.group_key == group_key,
            Alert.status.in_(("open", "ack")),
        )
        .order_by(Alert.created_at.desc())
```



```
        .limit(1)
    )
    .scalar_one_or_none()
)
```

Busca si ya existe una alerta abierta o reconocida para la misma regla y el mismo grupo.

Si existe, devuelve su `id`.

Si no existe, devuelve `None`.

Evitar duplicado

```
if existing_active_alert_id is not None:
    continue
```

Si ya hay una alerta activa, no crea otra.

Esto reduce ruido.

Ejemplo:

```
Ya hay una alerta open para:
rule_id = 2
group_key = "server-01"
```

Si llega otro evento que coincide con la misma regla y grupo, el sistema no genera una alerta duplicada.

Bloque threshold

```
if rule.threshold_count is not None and rule.threshold_seconds is not None:
```

Este bloque se aplica si la regla tiene configurado `threshold`.

Para que tenga sentido, deben existir ambos campos:

```
threshold_count
threshold_seconds
```

Ejemplo:

```
5 eventos en 60 segundos
```

Threshold requiere `group_key`

```
if group_key is None:
    continue
```

Si no hay `group_key`, no se puede aplicar `threshold`.

Esto tiene sentido porque el `threshold` está diseñado para agrupar eventos por una entidad, como `host`.

Cálculo de ventana temporal

```
window_start = now - timedelta(seconds=rule.threshold_seconds)
```

Calcula el inicio de la ventana temporal.

Ejemplo:

```
now = 12:00:00
threshold_seconds = 60
window_start = 11:59:00
```

Después se cuentan eventos desde ese momento.

Consulta base de conteo

```
stmt = select(func.count(Event.id)).where(Event.ts >= window_start)
```

Crea una consulta para contar eventos cuyo timestamp esté dentro de la ventana.

Conceptualmente:

```
SELECT COUNT(events.id)
FROM events
WHERE events.ts >= window_start;
```

Filtro por source

```
if rule.source:
    stmt = stmt.where(Event.source == rule.source)
```

Si la regla tiene origen definido, el conteo de threshold solo considera eventos de ese origen.

Filtro por severidad

```
if rule.severity_min is not None:
    stmt = stmt.where(Event.severity >= rule.severity_min)
```

Si la regla tiene severidad mínima, solo cuenta eventos que cumplan esa severidad.

Filtro por texto

```
if rule.contains:
    stmt = stmt.where(Event.message.ilike(f"%{rule.contains}%"))
```

Si la regla tiene texto `contains`, filtra eventos cuyo mensaje contenga ese texto.

`ilike` hace una búsqueda no sensible a mayúsculas/minúsculas.

Ejemplo:

```
Event.message ILIKE '%failed login%'
```

Filtro por meta_match

```
if rule.meta_match:
    stmt = stmt.where(Event.meta.contains(rule.meta_match))
```

Si la regla tiene `meta_match`, filtra eventos cuyo `meta` contenga esos pares clave-valor.

Esto usa capacidades JSONB de PostgreSQL.

Filtro por host

```
stmt = stmt.where(Event.meta.contains({"host": group_key}))
```

Añade un filtro para que solo cuente eventos del mismo host o grupo.

Ejemplo:

```
{
    "host": "server-01"
}
```

Esto conecta el threshold con `group_key`.

Ejecución del conteo

```
matched_count = db.execute(stmt).scalar_one()
```

Ejecuta la consulta y obtiene el número de eventos que cumplen las condiciones.

Ejemplo:

```
matched_count = 4
```

Comparación con `threshold_count`

```
if matched_count < rule.threshold_count:
    continue
```

Si el número de eventos encontrados es menor que el umbral requerido, no se genera alerta.

Ejemplo:

```
threshold_count = 5
matched_count = 4
```

Resultado:

```
4 < 5 → continue
```

Si `matched_count` es igual o mayor, la regla supera el threshold y puede generar alerta.

Creación de alerta

```
alert = Alert(
    rule_id=rule.id,
    event_id=ev.id,
    title=f"Rule matched: {rule.name}",
    group_key=group_key,
)
```

Crea un objeto `Alert`.

Campos:

```
rule_id    → id de la regla que coincidió
event_id   → id del evento recibido
title      → título generado automáticamente
group_key  → clave de agrupación
```

La alerta queda vinculada a la regla y al evento.

Añadir alerta a la sesión

```
db.add(alert)
```

Añade la alerta a la sesión SQLAlchemy.

Todavía no se confirma en la base de datos hasta que se ejecute:

```
db.commit()
```

Confirmar transacción

```
db.commit()
```

Confirma todos los cambios realizados durante la ingesta.

Puede incluir:

- Inserción del evento.
- Inserción de una o varias alertas.

Esto es importante: evento y alertas se guardan dentro de la misma transacción.

Devolver evento

```
return ev
```

Devuelve el objeto `Event` creado.

Como el endpoint tiene:

```
response_model=EventOut
```

FastAPI transforma `ev` en una respuesta JSON usando el schema `EventOut`.

Captura de excepciones

```
except Exception as e:
```

Captura cualquier excepción producida durante la ingesta.

La variable `e` contiene el error original.

Rollback

```
db.rollback()
```

Revierte los cambios pendientes en la transacción.

Esto evita guardar datos parcialmente si algo falla.

Ejemplo:

```
Evento insertado en sesión
Error creando alerta
Rollback
No queda una ingesta parcial inconsistente
```

Lanzar error HTTP

```
raise HTTPException(status_code=500, detail="Ingest failed") from e
```

Devuelve un error HTTP 500 al cliente.

Desglose:

```
status_code=500
```

Indica error interno del servidor.

```
detail="Ingest failed"
```

Mensaje devuelto al cliente.

```
from e
```

Mantiene la relación con la excepción original para depuración interna.

Resultado final del archivo

Este archivo expone:

```
POST /ingest
```

Su comportamiento completo es:

1. Recibe un payload validado.
2. Crea un evento.
3. Lo guarda temporalmente.
4. Calcula group_key.
5. Recupera reglas activas.
6. Evalúa criterios.
7. Aplica throttle.
8. Evita duplicados activos.
9. Evalúa threshold.
10. Crea alertas.
11. Hace commit.
12. Devuelve el evento.

7 Relación con el flujo técnico del laboratorio

`ingest.py` representa el flujo principal del SIEM MVP.

La relación completa es:

```
Cliente
  ↓
POST /ingest
  ↓
IngestPayload
  ↓
Event
  ↓
PostgreSQL
  ↓
Rule.enabled == True
  ↓
source / severity / contains / meta_match
```

```
↓
throttle / anti-duplicado / threshold
↓
Alert
↓
EventOut
```

Este archivo conecta los módulos más importantes del proyecto:

```
API
Base de datos
Modelos ORM
Schemas Pydantic
Motor de reglas
Gestión de alertas
```

8 Errores típicos o puntos importantes

db.flush() es necesario para obtener **ev.id**

La alerta necesita:

```
event_id=ev.id
```

Por eso se ejecuta:

```
db.flush()
```

antes de crear alertas.

group_key depende de **meta.host**

El sistema calcula:

```
ev.meta.get("host")
```

Si el evento no tiene `meta.host`, `group_key` será `None`.

Esto afecta directamente a:

```
throttle
anti-duplicado
threshold
```

Si `group_key` es `None`, no se aplica throttle

El propio código lo indica:

```
si group_key es None, NO aplicamos throttle
```

Esto evita limitar alertas cuando no hay una agrupación fiable.

Si `group_key` es `None`, no se aplica anti-duplicado

También por decisión de diseño, si no hay grupo, no se busca alerta duplicada.

Esto puede generar más alertas, pero evita deduplicar incorrectamente eventos no agrupados.

Threshold requiere `threshold_count` y `threshold_seconds`

El threshold solo se evalúa si existen ambos campos:

```
rule.threshold_count is not None
rule.threshold_seconds is not None
```

Si falta uno, la regla se trata como regla simple.

Threshold requiere group_key

Si una regla tiene threshold pero el evento no tiene group_key , se descarta:

```
if group_key is None:
    continue
```

Las alertas cerradas no bloquean nuevas alertas

Las consultas de throttle y anti-duplicado solo miran:

```
open
ack
```

No tienen en cuenta:

```
closed
```

Por tanto, si una alerta está cerrada, el sistema puede generar una nueva alerta para el mismo grupo.

El except oculta el error concreto al cliente

El cliente solo recibe:

```
{
  "detail": "Ingest failed"
}
```

Esto es más limpio, pero para depurar hay que mirar logs del backend.

9 Comandos útiles relacionados

Probar ingesta básica:

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 4,
  "message": "Failed login attempt for user admin",
  "meta": {
    "host": "server-01",
    "user": "admin",
    "ip": "192.168.1.10"
  }
}'
```

Probar ingesta sin meta :

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
```

```
"source": "auth",  
"severity": 2,  
"message": "Simple event without metadata"  
'}
```

Listar eventos después de ingestar:

```
curl http://localhost:8000/events
```

Consultar métricas:

```
curl http://localhost:8000/metrics
```

Consultar alertas:

```
curl http://localhost:8000/alerts
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Comprobar eventos en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, source, severity, message, meta, created_at  
FROM events ORDER BY id DESC LIMIT 10;"
```

Comprobar alertas en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, rule_id, event_id, title, group_key, status,  
created_at FROM alerts ORDER BY id DESC LIMIT 10;"
```

Comprobar reglas activas:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, enabled, source, severity_min,  
contains, throttle_seconds, threshold_count, threshold_seconds, meta_match FROM rules WHERE enabled IS  
TRUE;"
```

Probar importación del router:

```
docker exec -it siem-api python -c "from app.api.routes.ingest import router; print(router)"
```

Probar importación de la función auxiliar:

```
docker exec -it siem-api python -c "from app.api.routes.ingest import _compute_group_key;  
print(_compute_group_key)"
```